

claude-windows / c7679509-ee2b-4443-b240-0b3b1b1a7291

Metadata

```
- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c7679509-ee2b-4443-b240-0b3b1b1a7291\subagents\workflows\wf_a2b97cff-dcd\agent-a2b0b89e927cecf73.jsonl`
- SHA-256: `56fbb4e90acaa86b8ccde677e8661a8541b37b7c3e8ae890a778c67e25c5d9b`
- Source modified: `2026-06-21T14:01:04+00:00`
- Imported at: `2026-07-05T16:48:23+00:00`
- project: `wf_a2b97cff-dcd`
- session_id: `c7679509-ee2b-4443-b240-0b3b1b1a7291`
```

Transcript

1. user (2026-06-21T13:57:21.626Z)

Adversarially VERIFY the load-bearing claims below about packaging/distributing our Python app. For EACH claim, try to REFUTE it (check feasibility, licensing accuracy, version facts, whether it actually applies to a torch+opencv+ffmpeg+FastAPI app). Default to "uncertain" if you cannot confirm. Use web search to check current facts.

TOPIC: frozen-binaries

CLAIMS:

```
[
  {
    "claim": "The engine launches uvicorn single-process with the app OBJECT (not an import string), hard-bound to 127.0.0.1, with no workers=/reload= ? already matching the PyInstaller-recommended pattern (reload=False, workers=1, freeze_support()).",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/main.py:735 cross-checked with https://github.com/Kludex/uvicorn/discussions/1820 and https://pyinstaller.org/en/stable/common-issues-and-pitfalls.html"
  },
  {
    "claim": "The HARD blocker is the second Python env: WASB shells into a SEPARATE interpreter/env (WSL conda activate wasb, torch 1.11). A frozen single binary embeds ONE interpreter with no external python/conda, so the full-local WASB tier can't be one frozen process ? it needs a sidecar artifact or an ONNX replacement.",
    "confidence": "high",
    "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/backend/pipeline/detectors/wasb_runner.py:6,35-36,74 and native_wasb_runner.py:117; PyInstaller embedded-interpreter constraint https://github.com/orgs/pyinstaller/discussions/6046"
  },
  {
    "claim": "PyInstaller's license permits shipping a closed/any-licensed frozen app without distributing your source or PyInstaller's license (GPL-2.0 WITH bootloader exception; a few files Apache-2.0).",
    "confidence": "high",
    "source": "https://pyinstaller.org/en/stable/license.html"
  },
  {
    "claim": "Open-source Nuitka core is Apache-2.0 (PyPI); compiled output never inherits Nuitka's license; Nuitka Commercial is an OPTIONAL paid add-on, not required to distribute binaries.",
    "confidence": "medium",
    "source": "https://pypi.org/project/Nuitka/ and https://nuitka.net/doc/commercial-license.html and https://en.wikipedia.org/wiki/Nuitka"
  },
  {
    "claim": "A torch+opencv frozen bundle is typically 250 MB to 1 GB+ before model weights;
```

```

CUDA wheels push higher. Mitigate with opencv-python-headless, excluding
torch.cuda/torchvision/matplotlib/scipy, or ONNX-export + onnxruntime instead of torch.",
  "confidence": "high",
  "source": "https://www.codegenes.net/blog/pyinstaller-pytorch/ and
https://github.com/marcelotduarte/cx_Freeze/issues/2412 and
https://pydevtools.com/handbook/explanation/how-do-i-ship-a-python-application-to-end-users/"
},
{
  "claim": "PyInstaller --onefile commonly triggers AV/Windows-Defender false positives
(runtime self-extract to temp = packer-like); --onedir avoids extraction and trips far fewer
heuristics. Code-signing whitelists your cert and dramatically reduces (not eliminates) flags.",
  "confidence": "high",
  "source": "https://www.pythonguis.com/faq/problems-with-antivirus-software-and-pyinstaller/
and https://github.com/pyinstaller/pyinstaller/issues/6754"
},
{
  "claim": "macOS notarization of a frozen Python app requires Developer-ID signing + Hardened
Runtime + entitlements (allow-jit, allow-unsigned-executable-memory, disable-library-validation)
+ notarytool submit + stapler; --deep discouraged, sign inside-out. Windows needs Authenticode
(EV cert for clean SmartScreen).",
  "confidence": "high",
  "source": "https://developer.apple.com/forums/thread/744471 and
https://github.com/pyinstaller/pyinstaller/issues/7937"
},
{
  "claim": "Nuitka builds are long (5-15 min vs PyInstaller ~30s) and torch standalone is a
known heavy case, but Nuitka ships built-in PyTorch/onnxruntime rules (operator-kernel pruning)
that can shrink the torch payload.",
  "confidence": "high",
  "source": "https://pydevtools.com/handbook/reference/nuitka/ and
https://github.com/Nuitka/Nuitka/issues/754 and https://github.com/Nuitka/Nuitka/issues/2218"
},
{
  "claim": "torch needs explicit PyInstaller help: --copy-metadata torch plus hidden imports
like torch._C._jit/torch._C._nvrtc; onefile produces 'file already exists' duplication warnings
for torch/_C pyd.",
  "confidence": "high",
  "source": "https://github.com/pyinstaller/pyinstaller/discussions/7621 and
https://github.com/pyinstaller/pyinstaller/issues/8348"
},
{
  "claim": "Owner decision D10 SUPERSEDED the desktop-app plan: deliver a self-hosted
webserver + UI the user spins up (compute/storage as settings), so freezing should target a
CLI/server appliance, not a GUI desktop bundle.",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/khelsutra/docs/LOCAL_APPLIANCE_ARCHITECTURE.md:39 which
states it supersedes C:/Users/avidu/Projects/badminton-highlight-
indexer/docs/DESKTOP_APP_PLAN.md"
},
{
  "claim": "ffmpeg/ffprobe are system (non-Python) deps no freezer bundles automatically; they
must ship as side-by-side binaries, and to stay MIT/Apache/BSD-clean must be an LGPL build
(openh264/HW encoders, NOT GPL libx264).",
  "confidence": "high",
  "source": "C:/Users/avidu/Projects/badminton-highlight-indexer/docs/DESKTOP_APP_PLAN.md:64
(D4) and backend/pipeline/stitcher/compiler.py"
}
]

```

RECOMMENDATION UNDER TEST: VIABLE ? but ONLY as a CLI/server appliance (matching owner decision D10), tiered, and NOT as a single self-extracting onefile. (1) GEMINI-ONLY / no-torch tier = VIABLE NOW, low-risk: without torch the dependency set (fastapi, uvicorn, opencv-python-headless, numpy, pydantic, google-genai, supervision) freezes cleanly; bundle stays modest (tens of MB + ffmpeg). Use PyInstaller ONEDIR (not onefile ? avoids AV false positives), ship ffmpeg LGPL binaries side-by-side, wrap the existing indexer/backend.main --server entry (already freeze-friendly: single-process uvicorn on 127.0.0.1, main.py:735). This is the recommended

FIRST shippable artifact. (2) FULL LOCAL-MODEL tier (WASB) = PARTIALLY viable / higher-risk, defer. The blocker is NOT the freezer ? it is the dual-torch-env design: WASB runs in a separate torch-1.11 conda/WSL interpreter the engine shells out to (wasb_runner.py/native_wasb_runner.py). No freezer bundles two interpreters in one process. Two honest paths: (a) freeze the WASB env as its OWN sidecar binary spawned as a subprocess (doable, but doubles the torch payload to ~1GB+ and doubles signing/AV surface), or (b) the strategically right move ? finish the de-WSL native runner AND export WASB to ONNX, ship ONE onnxruntime (no torch), collapsing to a single freezable env and slashing size. Path (b) ties to the existing OWNED_MODEL_IMPLEMENTATION_PLAN export-clean mandate and the DESKTOP_APP_PLAN P0 spike. Tooling pick: PyInstaller (onedir) as default ? biggest recipe base for this exact stack, clean GPL-with-exception license, fast builds. Keep Nuitka (Apache-2.0, built-in torch/onnxruntime pruning) as fallback if size/IP-hardening matters once ONNX lands. Reject cx_Freeze (documented torch DLL pain, thinner recipes) and Briefcase (explicitly weak on C-extension/ML deps). Regardless of tool: budget per-OS code-signing (Win Authenticode/EV, Mac Developer-ID + Hardened Runtime + notarytool + stapler) and a license audit (LGPL ffmpeg only) as separate non-trivial workstreams.

2. assistant (2026-06-21T13:57:24.169Z)

I'll verify each claim adversarially. Let me start by examining the actual codebase files referenced in the claims, then check the licensing/version facts via web search.

3. user (2026-06-21T13:57:25.605Z)

```

700         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
701         segmenter = segmenter_cls(db_instance, global_config)
702         result = segmenter.process_video(video_id, local_video,
max_frames=args.max_frames)
703         segmentation_ran = True
704
705         if isinstance(result, Failure):
706             print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")
707             print("[FAIL] Indexing halted for video.")
708             segments = []
709         else:
710             segments = result.value if hasattr(result, "value") else result
711             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
712             # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
713             _finalize_reingest(video_id, segments, play_wm, cand_wm)
714         finally:
715             # Always emit the per-video observability report. Never raises.
716             paths = emit_indexer_report(
717                 db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
718                 val_results=val_results, val_context=val_context,
719                 segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
720                 was_reingest=was_reingest, segmentation_ran=segmentation_ran,
721             )
722         if paths:
723             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
724             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
725             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
726         return
727
728         # Start Local Server Mode
729         if args.server or not args.video_path:
730             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
731             # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
732             # in-process wake-timer for any parked 'waiting' job died with the old process,
so
733             # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
```

```

734         _get_executor().submit(_drain_due_jobs)
735         uvicorn.run(app, host="127.0.0.1", port=args.port)
736
737     if __name__ == "__main__":
738         # `python -m backend.main` runs THIS file as `__main__`, but `job_worker`/`uploads`
739         # shared mutable state (`db_instance`, `global_config`) via `import backend.main` ?
740         # module object. If `main()` runs here it configures `__main__`'s globals, leaving
741         # `backend.main.db_instance` = None; the async-job drain then crashes silently on
742         # `None.claim_due_job()` and every submitted job hangs in 'queued' forever. Dispatch
743         # `backend.main` so `main()` configures the module everyone actually reads. (No
744         # imported as `backend.main`, `__name__` != "__main__", so this block doesn't re-
745         # fire.)
746         import backend.main as _bm
747         _bm.main()
748

```

4. user (2026-06-21T13:57:25.911Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4     1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5     2. copies wasb_infer.py into the WASB repo's src/ dir,
6     3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7     4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex
14    import logging
15    import subprocess
16    from dataclasses import dataclass
17    from typing import Dict, Any, Optional, Tuple
18
19    # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20    # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
21    TrackNetRunner.
22    from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
23    from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
24    wsl_tilde_quote
25
26    logger = logging.getLogger(__name__)
27
28    # Windows path to the inference wrapper we stage into WSL.
29    _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
30
31    @dataclass
32    class WasbConfig:
33        """Resolved settings for a WASB WSL run (built from config.json ->
34        indexing.wasb)."""
35        distro: str = "Ubuntu"
36        repo_dir: str = "~/models/WASB-SBDT"
37        conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
38        conda_env: str = "wasb"
39        weights_path: str = "~/models/WASB-
40        SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
41        sport: str = "badminton"

```

```

39     wsl_stage_dir: str = "~/clips"
40     timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41     keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42     min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43     # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44     # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45     # extraction that dominates a long clip. Output is bit-identical to the PNG path
46     # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47     stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg
71
72         def healthcheck(self) -> Tuple[bool, str]:
73             """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74             cmd = (f"conda activate {self.cfg.conda_env} && "
75                  f"python -c \"import torch; print('torch', torch.__version__, "
76                  f"'cuda', torch.cuda.is_available())\"")
77             try:
78                 res = self._wsl_bash(cmd)
79             except FileNotFoundError:
80                 return False, "`wsl` not found ? Windows + WSL2 required."
81             except subprocess.TimeoutExpired:
82                 return False, "WSL healthcheck timed out."
83             out = (res.stdout or "").strip()
84             if res.returncode != 0:
85                 return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86             return True, out
87
88         def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89             src_mnt = to_wsl_mnt_path(video_win_path)
90             reason = self.wsl_source_unreadable_reason(src_mnt)
91             if reason:
92                 logger.error("Cannot stage video into WSL: %s", reason)
93                 return None
94             dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95             cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
96             try:
97                 res = self._wsl_bash(cmd)
98             except subprocess.TimeoutExpired:

```

```

99         logger.error("Staging video into WSL timed out.")
100         return None
101     if res.returncode != 0 or "OK" not in (res.stdout or ""):
102         logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
103         return None
104         return dest
105
106     def _stage_infer_script(self) -> bool:
107         """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
108         src_mnt = to_wsl_mnt_path(_INFER_WIN)
109         cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
OK"
110         res = self._wsl_bash(cmd)
111         return res.returncode == 0 and "OK" in (res.stdout or "")
112
113     def run_predict(self, video_win_path: str, output_win_dir: str,
114                     video_id: Optional[str] = None) -> Optional[str]:
115         """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
116
117         All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
118         two videos that share a filename (e.g. two ``match.mp4`` from different folders)
119         can never reuse each other's staged frames, resumable cache, or CSV.
120
121         # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
122         # translation: to_wsl_mnt_path passes relative paths through unchanged, so
123         # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
124         # the CSV landed inside WSL while Windows polled the repo-relative path.
125         output_win_dir = os.path.abspath(output_win_dir)
126         os.makedirs(output_win_dir, exist_ok=True)
127         # Normalize for cross-platform basename (tests use Windows paths on Linux).
128         video_win_path = str(video_win_path).replace("\\", "/")
129         base = os.path.basename(video_win_path)
130         stem, ext = os.path.splitext(base)
131         # Unique, content-derived key: same filename + different bytes -> different key.
132         key = f"{stem}__{video_id[:12]}" if video_id else stem
133         wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134         expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136         if not self._stage_infer_script():
137             logger.error("Could not stage wasb_infer.py into WSL.")
138             return None
139         staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140         if staged_video is None:
141             return None
142         frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144         # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145         # WSL stage filesystem is already low so the user can clean up before it fills.
146         # (Streaming never materializes the PNGs, so the guard only matters off the fast
path.)
147         if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
148             free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
149             if free is not None and free < self.cfg.min_free_gb:
150                 logger.warning(
151                     "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
extraction "
152                     "may fill the disk ? consider running tools/cleanup_caches.py.",
153                     free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
154
155         # Fast path: stream-decode the video in-process (no PNG extraction). Output is
156         # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
clip.
157         if self.cfg.stream_video:
158             frame_args = "--stream-video "

```

```

159         else:
160             frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
161             cmd = (
162                 f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
163                 f"python wasb_infer.py "
164                 f"--video {wsl_tilde_quote(staged_video)} "
165                 f"{frame_args}"
166                 f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
167                 f"--sport {shlex.quote(self.cfg.sport)} "
168                 f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
169             )
170             logger.info("Running WASB inference on %s ...", base)
171             try:
172                 res = self._wsl_bash(cmd)
173             except subprocess.TimeoutExpired:
174                 logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
175                 return None
176             if res.returncode != 0:
177                 logger.error("WASB inference failed:\n%s", (res.stderr or
178 res.stdout)[-2000:])
179                 return None
180             if not os.path.exists(expected_csv_win):
181                 logger.error("WASB finished but expected CSV not found at %s",
182 expected_csv_win)
183                 return None
184             logger.info("WASB trajectory CSV: %s", expected_csv_win)
185
186             # Success ? the trajectory CSV is the durable output; the staged video copy and
187             # the (huge) decoded frame cache are now pure intermediates. Delete them unless
188             # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
189             # so the frame cache + detector cache remain for a resumable re-run.
190             # (Streaming never writes a frame cache, so there's only the staged video to
191 drop.)
192             if not self.cfg.keep_frames:
193                 stale = [staged_video] if self.cfg.stream_video else [frames_dir,
194 staged_video]
195                 logger.info("Cache hygiene: removing staged video%s for %s "
196                             "(set indexing.wasb.keep_frames=true to retain).",
197                             "" if self.cfg.stream_video else " + frame cache", key)
198                 self._wsl_rm_rf(stale)
199                 return expected_csv_win
200
201             # Trajectory parsing + windowing are inherited from tracknet_runner (model-
202 agnostic).
203             parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
204             trajectory_to_action_windows =
205 staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

5. assistant (2026-06-21T13:57:30.081Z)

Claim 1 partially verified (main.py:735 confirms `uvicorn.run(app, host="127.0.0.1", port=args.port)` ? app object, no workers/reload). Note: there is NO `freeze\_support()` call visible. Let me check for that and read the other referenced files.

6. user (2026-06-21T13:57:31.067Z)

```

1         """Linux-native WASB shuttle-trajectory runner (no WSL).
2
3         The de-risk goal the ``DetectorRunner`` ABC was built for (see ``base.py`` ?
4         "Optional Linux-native path"; ``docs/DECOUPLED_COMPUTE/04-BUILD-AND-TEST-PLAN.md``
5 M3.5).
6         It is the sibling of :class:`~backend.pipeline.detectors.wasb_runner.WasbRunner` for a
7         Linux GPU box (GCP / a SaaS worker): it runs the same verified ``wasb_infer.py``
8         core

```

```

7         that the WSL adapter runs, so the trajectory CSV is identical (to a sub-pixel float
tolerance;
8         see the acceptance gate below) ? but WITHOUT the WSL transport:
9
10        * NO ``wslCommandMixin`` (no ``wsl -d <distro> bash -c``), NO ``conda activate`` (the
11        ``wasb`` env's python is invoked **by path**), NO ``/mnt/c`` path translation
12        (``to_wsl_mnt_path``). Paths are native; the video is already local on the box.
13        * ``device`` is ``cuda`` (default ? the WSL parity path), ``mps``, ``cpu``, or
``auto``
14        (M4: cuda-if-available-else-cpu ? the CPU fallback for a no-GPU box; explicit
``cuda`` stays
15        strict); threaded into ``wasb_infer.py --device`` (and so the Hydra
``runner.device`` override).
16        * weights / repo / python / scratch come from the **environment** first
17        (``WASB_WEIGHTS`` / ``WASB_REPO_DIR`` / ``WASB_PYTHON`` / ``WASB_SCRATCH``), then
18        ``indexing.wasb.*`` config, then a default ? so a box needs no config edits.
19
20        It emits the identical ``Frame,Visibility,X,Y`` CSV (same ``{stem}__{vid12}_wasb.csv``
21        name as the WSL runner) and reuses the model-agnostic ``parse_trajectory_csv`` /
22        ``trajectory_to_action_windows`` from ``tracknet_runner``, so the trajectory hybrids and
23        the windowing brain are unchanged. Selected via the existing ``detector_impl`` seam
24        (``trajectory_hybrid._resolve_runner`` ? ``_build_native_runner``);
``detector_impl="native"``.
25
26        **Acceptance gate (parity, NOT strict byte-equality):** the SAME clip through the WSL
runner
27        (Windows) and this runner (Linux, ``device=cuda``) yields the same trajectory CSV + the
same
28        candidate windows **within a small float tolerance (~1e-3 px)**. cuDNN is non-
deterministic
29        ACROSS GPUs, so a sub-pixel diff is expected on different hardware ? but it is byte-
exact
30        run-to-run on a FIXED GPU. ? Validated 2026-06-20 on an RTX 2070 (via WSL): native
streaming
31        vs the cached WSL-disk CSV matched **1194/1195** frames (the lone diff a ~5e-5 px edge
32        artifact), and two native runs were byte-identical (deterministic). This is a hardware
check
33        (a real GPU + the ``wasb`` env), so the offline, subprocess-mocked suite asserts the
contract,
34        not the numbers.
35        """
36        import os
37        import shutil
38        import logging
39        import subprocess
40        from dataclasses import dataclass
41        from typing import Any, Dict, List, Optional, Tuple
42
43        from backend.pipeline.detectors.base import DetectorRunner
44        from backend.pipeline.detectors.device import AUTO, VALID_DEVICES, DeviceContext
45        # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical to
WSL/TrackNet.
46        from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
47
48        logger = logging.getLogger(__name__)
49
50        # Path to the inference core we copy into the WASB repo's src/ (so it can import WASB's
51        # detectors/trackers/dataloaders + find configs/), exactly as the WSL adapter does.
52        _INFER_PY = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
53
54        # Single torch/cuda probe code, used by BOTH healthcheck and the lazy device-resolution
probe
55        # so the "cuda True/False" string they parse can never drift apart.
56        _CUDA_PROBE_CODE = "import torch; print('torch', torch.__version__, 'cuda',
torch.cuda.is_available())"

```



```

57
58
59 @dataclass
60 class NativeWasbConfig:
61     """Resolved settings for a Linux-native WASB run.
62
63     Built via :meth:`from_indexing_cfg` with **environment overrides** taking precedence
64     over ``indexing.wasb.*`` config (the box sets env; no config edits needed).
65     ``device`` defaults to ``cuda`` (the WSL parity path); ``stream_video`` defaults to True (the
66     perf win).
67     """
68     repo_dir: str = "~/models/WASB-SBDT" # native clone of nttcom/WASB-SBDT
69     python_bin: str = "python" # the `wasb` conda env python (invoked by
70     path, no activate)
71     weights_path: str = "" # REQUIRED (env WASB_WEIGHTS or
72     indexing.wasb.weights_path)
73     sport: str = "badminton"
74     device: str = "cuda" # auto | cuda | mps | cpu (auto = cuda-if-
75     available-else-cpu)
76     scratch_dir: str = "" # frame-extraction scratch (env); "" = next
77     to the output dir
78     timeout_sec: int = 1800 # 30 min; a hung GPU call is killed (0 = no
79     timeout)
80     keep_frames: bool = False # retain the decoded frame cache after
81     success (debug only)
82     # NATIVE DEFAULT = STREAMING: skip the cv2 PNG extraction that dominates a long
83     clip's
84     # wall-clock (measured ~50s on a 20s clip) ? bit-identical to disk (verified on a
85     real GPU
86     # 2026-06-20). The GPU box reads from a local NVMe so streaming is the right default
87     here.
88     stream_video: bool = True
89
90     @classmethod
91     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "NativeWasbConfig":
92         w = (idx_cfg or {}).get("wasb", {}) or {}
93         env = os.environ.get
94         # stream_video is tri-state in config: None/absent => native default (stream);
95         an
96         # explicit True/False still wins (e.g. False for a container cv2 can't seek).
97         sv = w.get("stream_video")
98         return cls(
99             repo_dir=env("WASB_REPO_DIR") or w.get("repo_dir", "~/models/WASB-SBDT"),
100             python_bin=env("WASB_PYTHON") or w.get("python_bin", "python"),
101             weights_path=env("WASB_WEIGHTS") or w.get("weights_path", "") or "",
102             sport=w.get("sport", "badminton"),
103             device=env("WASB_DEVICE") or w.get("device", "cuda"),
104             scratch_dir=env("WASB_SCRATCH") or env("RALLY_SCRATCH") or env("TMPDIR") or
105             "",
106             timeout_sec=int(w.get("timeout_sec", 1800)),
107             keep_frames=bool(w.get("keep_frames", False)),
108             stream_video=(True if sv is None else bool(sv)),
109         )
110
111 class NativeWasbRunner(DetectorRunner):
112     """WASB runner that executes natively on a Linux GPU box ? no WSL. See module
113     docstring."""
114
115     def __init__(self, cfg: NativeWasbConfig):
116         self.cfg = cfg
117         # Cached CUDA-availability probe (used to resolve device="auto"). Set by
118         healthcheck;
119         # lazily probed by run_predict if healthcheck was skipped. None = not yet

```

```

probed.
107         self._cuda_available: Optional[bool] = None
108
109         # --- low-level native exec (the single place tests patch)
110         -----
111         def _run(self, argv: List[str], cwd: Optional[str] = None) ->
subprocess.CompletedProcess:
112             logger.debug("native exec: %s (cwd=%s)", " ".join(argv), cwd)
113             return subprocess.run(argv, cwd=cwd, capture_output=True, text=True,
114                                   timeout=(self.cfg.timeout_sec or None))
115
116         # --- device resolution (M4 DeviceContext: device="auto" ? cuda-if-available-else-
cpu) -----
117         def _probe_cuda(self) -> bool:
118             """Probe ``torch.cuda.is_available()`` in the wasb env's python (a subprocess).
Any failure
119             (missing python / timeout / import error) is treated as 'no CUDA'."""
120             try:
121                 res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
122             except (FileNotFoundError, subprocess.TimeoutExpired):
123                 return False
124             return res.returncode == 0 and "cuda True" in (res.stdout or "")
125
126         def _cuda_is_available(self) -> bool:
127             """CUDA availability, cached across healthcheck + run_predict so we probe at
most once."""
128             if self._cuda_available is None:
129                 self._cuda_available = self._probe_cuda()
130             return self._cuda_available
131
132         def _effective_device(self) -> str:
133             """The torch device to actually pass to ``wasb_infer.py``. Only ``auto`` needs
the CUDA
134             probe; an explicit cuda/mps/cpu resolves to itself (so ``device='cuda'`` is
unchanged)."""
135             if self.cfg.device == AUTO:
136                 return DeviceContext(AUTO, self._cuda_is_available()).effective
137             return self.cfg.device
138
139         def _repo_src(self) -> str:
140             return os.path.join(os.path.expanduser(self.cfg.repo_dir), "src")
141
142         def _copy_infer_script(self) -> bool:
143             """Copy wasb_infer.py into the WASB repo src/ so it imports WASB modules + finds
configs/."""
144             repo_src = self._repo_src()
145             try:
146                 os.makedirs(repo_src, exist_ok=True)
147                 shutil.copy(_INFER_PY, os.path.join(repo_src, "wasb_infer.py"))
148                 return True
149             except OSError as e:
150                 logger.error("Failed to copy wasb_infer.py into %s: %s", repo_src, e)
151                 return False
152
153         def _rm_rf(self, path: Optional[str]) -> None:
154             """Best-effort recursive delete of a native path (post-success cleanup; never
raises)."""
155             if path:
156                 shutil.rmtree(path, ignore_errors=True)
157
158         def healthcheck(self) -> Tuple[bool, str]:
159             """Verify weights + repo present and the `wasb` python imports torch (and sees a
GPU on cuda)."""
160             if not self.cfg.weights_path:
161                 return False, ("WASB weights path is empty ? set WASB_WEIGHTS (or

```

```

indexing.wasb.weights_path).")
161         weights = os.path.expanduser(self.cfg.weights_path)
162         if not os.path.exists(weights):
163             return False, f"WASB weights not found at {weights}"
164         repo_src = self._repo_src()
165         if not os.path.isdir(repo_src):
166             return False, (f"WASB-SBDT repo src/ not found at {repo_src} ? clone
nttcom/WASB-SBDT "
167                             f"(set WASB_REPO_DIR / indexing.wasb.repo_dir).")
168         # Fail fast on a typo'd device (the device-policy seam) ? a clear error here
beats a cryptic
169         # argparse rc=2 from wasb_infer.py at run time.
170         if self.cfg.device not in VALID_DEVICES:
171             return False, (f"unknown device {self.cfg.device!r} ? valid: {'',
''.join(VALID_DEVICES)}").")
172         try:
173             res = self._run([self.cfg.python_bin, "-c", _CUDA_PROBE_CODE])
174         except FileNotFoundError:
175             return False, f"python binary not found: {self.cfg.python_bin!r} (set
WASB_PYTHON).")
176         except subprocess.TimeoutExpired:
177             return False, "native torch healthcheck timed out."
178         out = (res.stdout or "").strip()
179         if res.returncode != 0:
180             return False, f"`{self.cfg.python_bin}` torch import failed: {(res.stderr or
out).strip()}"
181         # M4: resolve the device. Cache the probe so run_predict reuses it (no second
subprocess).
182         self._cuda_available = "cuda True" in out
183         ctx = DeviceContext(self.cfg.device, self._cuda_available)
184         if ctx.cuda_required_but_missing:
185             # Unchanged pre-M4 behaviour for an EXPLICIT cuda request: hard-fail, never
a silent
186             # slow-CPU downgrade. Set indexing.wasb.device=auto for a CPU fallback.
187             return False, f"device=cuda but torch.cuda.is_available() is False ({out})"
188         if ctx.fell_back:
189             logger.warning("device=auto: no CUDA GPU detected (%s) ? FALLING BACK TO
CPU; WASB "
190                             "inference will be slow. Set indexing.wasb.device=cuda to
require a GPU.", out)
191         return True, out
192
193     def run_predict(self, video_win_path: str, output_win_dir: str,
194                    video_id: Optional[str] = None) -> Optional[str]:
195         """Run WASB natively on a video. Returns the path to the trajectory CSV, or
None.
196
197         Output artifacts are namespaced by ``video_id`` (the file MD5) exactly as the
WSL
198         runner does, and the CSV is named ``{stem}__{vid12}_wasb.csv`` ? byte-for-byte
the
199         same downstream contract, so the parity comparison is apples-to-apples.
200         """
201         output_dir = os.path.abspath(output_win_dir)
202         os.makedirs(output_dir, exist_ok=True)
203         # Normalize for a cross-platform basename (tests pass Windows paths on Linux).
204         norm = str(video_win_path).replace("\\", "/")
205         stem, _ext = os.path.splitext(os.path.basename(norm))
206         # Content-derived key: same filename + different bytes -> different key (no
cache reuse).
207         key = f"{stem}__{video_id[:12]}" if video_id else (stem or "wasb")
208         out_csv = os.path.join(output_dir, f"{key}_wasb.csv")
209
210         if not self._copy_infer_script():
211             logger.error("Could not stage wasb_infer.py into the WASB repo src/.")

```

```

212         return None
213
214     weights = os.path.expanduser(self.cfg.weights_path)
215     argv = [self.cfg.python_bin, "wasb_infer.py",
216             "--video", os.path.abspath(str(video_win_path))]
217     frames_dir: Optional[str] = None
218     if self.cfg.stream_video:
219         # Fast path: decode in-process, no PNG extraction. Output is bit-identical
to disk.
220         argv.append("--stream-video")
221     else:
222         scratch = self.cfg.scratch_dir or output_dir
223         frames_dir = os.path.join(scratch, f"{key}_frames")
224         argv += ["--frames_out_dir", frames_dir]
225         # M4: resolve device="auto" to the effective device (cuda-if-available-else-
cpu); an
226         # explicit cuda/mps/cpu passes through unchanged. Never sends "auto" to
wasb_infer.py.
227         device = self._effective_device()
228         argv += ["--weights", weights,
229                 "--sport", self.cfg.sport,
230                 "--device", device,
231                 "--out", out_csv]
232
233         logger.info("Running native WASB inference (device=%s%s) on %s ...",
234                     device, " [auto?fallback]" if self.cfg.device == AUTO and device !=
"cuda" else "",
235                     os.path.basename(norm))
236         try:
237             res = self._run(argv, cwd=self._repo_src())
238         except FileNotFoundError:
239             logger.error("python binary not found: %r (set WASB_PYTHON).",
self.cfg.python_bin)
240             return None
241         except subprocess.TimeoutExpired:
242             logger.error("WASB native inference timed out after %ss.",
self.cfg.timeout_sec)
243             return None
244         if res.returncode != 0:
245             logger.error("WASB native inference failed:\n%s", (res.stderr or res.stdout
or "")[-2000:])
246             return None
247         if not os.path.exists(out_csv):
248             logger.error("WASB native finished but expected CSV not found at %s",
out_csv)
249             return None
250         logger.info("WASB trajectory CSV: %s", out_csv)
251
252         # Success ? the CSV is the durable output; the decoded frame cache is a pure
intermediate.
253         # On any earlier failure we return above WITHOUT cleaning, so the cache survives
a resume.
254         # (Streaming never materializes frames, so there is nothing to drop on that
path.)
255         if not self.cfg.keep_frames and frames_dir:
256             logger.info("Cache hygiene: removing frame cache for %s "
257                         "(set indexing.wasb.keep_frames=true to retain).", key)
258             self._rm_rf(frames_dir)
259         return out_csv
260
261     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
262     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
263     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)

```

7. user (2026-06-21T13:57:31.956Z)

No matches found

8. assistant (2026-06-21T13:57:38.185Z)

Important finding: NO `freeze\_support()` exists in main.py. Claim 1 asserts the code "already matching the PyInstaller-recommended pattern (reload=False, workers=1, freeze_support())". The reload/workers parts are correct (uvicorn.run with app object, host 127.0.0.1, no workers/reload kwargs), but `freeze\_support()` is NOT present. Let me verify there's no multiprocessing freeze_support anywhere relevant.

Also, native_wasb_runner.py:117 is `\_probe\_cuda` ? it invokes `self.cfg.python\_bin` ("python" by default, or the wasb conda env python by path) as a SEPARATE subprocess. So claim 2's citation is accurate: WASB shells into a separate interpreter (torch). Let me check the referenced docs and verify the torch version claim.

9. user (2026-06-21T13:57:40.202Z)

No files found

10. user (2026-06-21T13:57:40.382Z)

```

1      # Cross-Platform Desktop App ? "Plug in your card, get your highlights"
2
3      > **Status:** `DRAFT` ? `owner-gated` (scoping doc only; **nothing is built**) .
**Owner:** avidullu . **Created:** 2026-06-18 . **Last updated:** 2026-06-18
4      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
DONE)
5      > **Tracking anchors:** $7 progress tracker is the source of truth; indexed in
`docs/README.md`; pointer in memory `current-state` (and `docs/NEXT_STEPS.md` once the owner
greenlights P0).
6      > **Relation to existing docs:** **peer-of**
[PLATFORM_ARCHITECTURE.md](PLATFORM_ARCHITECTURE.md) (this is the **`LocalDesktop`
ComputeBackend** made into a product) . **realizes the L0 tier** of
[VIDEO_LOCALITY_MODEL.md](VIDEO_LOCALITY_MODEL.md) . **consumes**
[OWNED_MODEL_IMPLEMENTATION_PLAN.md](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (the smaller/export-
clean model is one answer to the compute crux) . bound by
[COMMERCIALIZATION.md](COMMERCIALIZATION.md) licensing guardrails.
7      > **Honesty note:** every claim is tagged `[verified]` (checked against code/measured),
`[researched]` (needs sign-off), or `[design]` (proposed). **The entire build is `[design]`** ?
this doc scopes; it does not ship. Not legal advice.
8
9      ---
10
11     ## 0. TL;DR
12     Package the mature local pipeline (WASB shuttle detector + improved windowing + ffmpeg
stitch, already runnable fully-offline via [tools/generate_highlights.py --skip-ai-
handoff](../tools/generate_highlights.py)) as a **cross-platform desktop app** for **non-
technical recreational players**: plug in the camera's USB/SD card, click once, get back **only
the highlight reels** ? fully on-device, **no upload**, **no copy of the GBs of 4K originals**.
This is the product surface that fits the "local, cheap, efficient" thesis and the L0 ("nothing
leaves the machine") tier of `VIDEO_LOCALITY_MODEL.md`.
13
14     **The make-or-break unknown is compute on a typical enthusiast laptop** ? which often
has **no discrete GPU**. WASB runs at **~3.4x real-time on a laptop RTX 2070 Super** `[verified,
measured]`; on CPU it is far slower, and Apple Silicon has **no CUDA** at all. **P0 is a
feasibility spike that measures CPU and Apple-MPS throughput for a ~10-min clip before any app
is built.** If CPU-only is unacceptable on commodity hardware, the path needs a smaller/faster
owned model (ties `OWNED_MODEL_IMPLEMENTATION_PLAN`) or a tiered "fast-if-GPU, slow-but-works-
if-CPU" design. **Everything past P0 is gated on that result.**
15

```

```

16      ## 1. Problem & goal
17
18      **Why now / the user.** Target users are **recreational sports enthusiasts, not tech-savvy**. Their match videos are **huge** (GBs of 4K). The cloud-upload product surface fails them on every axis: slow on a typical Indian uplink (\VIDEO_LOCALITY_MODEL.md` $0: 10 GB ? ~1 h at 20 Mbps, ~4.5 h at 5 Mbps), bandwidth-costly, privacy-fraught, and it bloats both our storage and their drives. The thing the customer actually wants back is **kilobytes of timestamps ? a highlight reel** (\VIDEO_LOCALITY_MODEL.md` $1: "timestamps are the product; the video is dead weight").
19
20      **The concrete outcome ("good" looks like this):**
21      1. User plugs the GoPro/phone SD card or camera USB into their laptop.
22      2. The app detects the card, lists the match videos on it.
23      3. User clicks **Generate Highlights**.
24      4. The app makes an on-device proxy, runs WASB detection + windowing + stitches the reel ? **100% locally**.
25      5. The reel(s) land in a user-chosen folder. **The originals are never copied off the card and never bloat any drive.**
26
27      No account, no upload, no cloud dependency in the default path. Gemini stays an **optional** "better brain if you want it" toggle ? never required, never bundled as a hard dependency, never a training source (\COMMERCIALIZATION` C4).
28
29      **Read to get current:** \VIDEO_LOCALITY_MODEL.md` (the L0/locality framing + OQ5 packaging seed), \PLATFORM_ARCHITECTURE.md` $3.2 (the ComputeBackend seam), [\backend/pipeline/detectors/base.py](\backend/pipeline/detectors/base.py) (the detector ABC that already anticipates a Linux-native runner), \tools/generate_highlights.py` (the local entrypoint this app wraps).
30
31      ## 2. Decisions locked
32
33      | # | Decision | Source / date | Implication |
34      |---|-----|-----|-----|
35      | D1 | **Default path is 100% on-device, no upload, no Gemini.** The reel is produced from the local pipeline (\--skip-ai-handoff`). | This doc · 2026-06-18; matches \VIDEO_LOCALITY_MODEL` L0 | The app must never *require* a network call. Gemini is an opt-in toggle only. |
36      | D2 | **Never copy the originals.** The app reads source video in place (off the card / a chosen folder) and writes **only** reels + a tiny report to a user folder. | This doc; mirrors \generate_highlights.py` non-destructive \_atomic_publish` discipline | No "import library" that duplicates 10 GB files. Detection runs on a proxy; stitch reads the original in place. |
37      | D3 | **Drop the WSL2 dependency; run the detector natively per-OS.** | This doc; enabled by the \DetectorRunner` ABC [verified] | Windows-native (no WSL), Linux+NVIDIA (CUDA directly), macOS (MPS or CPU). The WSL adapter stays for the current dev box but is **not** the shipped path. |
38      | D4 | **Commercial-clean licensing for everything bundled: MIT / Apache-2.0 / BSD only** (code, weights). ffmpeg ships as an **LGPL build** (no GPL-only encoders like x264). | \COMMERCIALIZATION` (ratified 2026-06-09); \VIDEO_LOCALITY_MODEL` OQ5 (**ffmpeg LGPL build = licensing-clean**) | WASB-SBDT weights are MIT [verified] (see \tracknet-wasb-wsl2-decision`). The stitch must use openh264 (BSD) or OS hardware encoders, **not** statically-linked x264. See $3.4. |
39      | D5 | **Reuse the existing pipeline verbatim; add only the shell + a native runner + device-I/O.** | \backend/pipeline/detectors/base.py` reuse contract [verified] | No re-architecture of windowing/stitch. New code = native runner, USB/SD detection, app shell, packaging. |
40      | D6 | **Feasibility is gated on P0.** No app shell, no packaging work starts until the compute spike returns a verdict. | This doc; \decision-rigor-norm` (NUMBERS INFORM, FOUNDATIONS DECIDE) | P0 is the only un-gated deliverable; P1?P3 are owner-greenlit *after* P0 numbers. |
41
42      ## 3. Foundation ? research / prior art [design] / [researched]
43
44      ### 3.1 The compute reality (the crux) ? what we know vs must measure
45      - **WASB on a discrete laptop GPU is acceptable** [verified, measured]: on the owner's **RTX 2070 Super Max-Q (8 GB)**, WASB inference is the pipeline driver at **~3.2?3.4× real-time** (?3.4 GPU-min per footage-min, ~0.055 s/frame), peak VRAM ~5.7?6.0 GB; proxy (NVENC)

```

```

~0.2x RT; 4K stitch (CPU libx264) ~16 s/rally; end-to-end ?5x clip length. Source: `current-
state.md` `COST_SCALING.md` checkpoint (6 Boxhill clips, 2026-06-18).
46 - **The typical enthusiast laptop has NO discrete GPU.** A 10-min 4K clip at 3.4x RT is
~34 GPU-min *with* a 2070S. On integrated graphics / CPU-only the wall-clock is unknown and
could be 10x+ worse ? possibly **hours per clip**, which would kill the one-click promise.
47 - **Apple Silicon has no CUDA.** WASB/TrackNet weights are PyTorch; PyTorch MPS (Metal)
backend *may* run them, but op-coverage and speed on the WASB model are **unverified**
`[design]`. CPU fallback on Apple is also unverified.
48 - **Owned-model tie-in.** `OWNED_MODEL_IMPLEMENTATION_PLAN.md` already states the on-
device answer: *train in Python ? **export across the boundary** (ONNX / Core ML / ExecuTorch /
GGUF) ? a thin device shell runs the artifact with zero Python*, with the standing constraint to
**train with export-clean ops**. A smaller, exported, quantized shuttle detector is a credible
answer if the CPU/MPS numbers on the current WASB weights are bad. **This makes P0 partly a
referendum on whether the desktop product needs the owned-model track.**
49
50     ### 3.2 The de-WSL native runner is already designed-for `[verified]`
51     The detector layer was deliberately built to make this port low-regret.
[`backend/pipeline/detectors/base.py`](../backend/pipeline/detectors/base.py) documents,
verbatim, an ***Optional Linux-native path (the main de-risk goal)***: a
`LinuxNativeRunner`/`ONNXRunner` that **Skip[s] all wsl / _stage / conda activate, load[s]
weights locally (torch or onnx)? write[s] the exact same CSV format so
`trajectory_to_action_windows` works verbatim.** The WSL plumbing lives in a separate
`WslCommandMixin`, explicitly so a native runner **must not inherit WSL plumbing.** The actual
inference math is already isolated in
[`wasb_infer.py`](../backend/pipeline/detectors/wasb_infer.py) (torch + the WASB-SBDT modules,
emitting `Frame,Visibility,X,Y`). **So the port is: lift `wasb_infer.py` out of WSL, load
weights with native torch/onnxruntime per-OS, keep the identical CSV contract.** The windowing
brain (`trajectory_to_action_windows`) and stitch are untouched.
52
53     ### 3.3 Compute backends to evaluate in P0
54     | Path | OS | Toolkit | Status |
55     |---|---|---|---|
56     | CUDA-native | Windows-native, Linux+NVIDIA | torch+CUDA (no WSL) | proven in WSL
`[verified]`; native = port |
57     | Apple MPS | macOS (Apple Silicon) | torch MPS / Core ML export | **unverified ? P0
must measure** |
58     | CPU | all (the no-GPU laptop) | torch-CPU / **onnxruntime** (often fastest on CPU) |
**unverified ? P0 must measure; the gating number** |
59     | Smaller/exported model | all | ONNX/Core ML/ExecuTorch, quantized | deferred to owned-
model track; P0 informs whether it's *needed* |
60
61     ### 3.4 App-shell prior art & licensing landscape `[researched]`
62     - **App shell options:** **Tauri** (Rust + system webview; ~3?10 MB binaries;
MIT/Apache) vs **Electron** (bundled Chromium; ~100?150 MB; MIT) vs **CLI + installer** (wrap
`generate_highlights.py`; smallest effort). All shell licenses are clean.
63     - **Backend packaging:** the Python/FastAPI backend bundles via **PyInstaller** or a
frozen venv as a local "sidecar." Packaging was already modernized ? **PEP 621 `pyproject.toml`
+ an `indexer` console entry point** shipped (`DEFERRED_HARDENING_PLAN` #7, DONE, PR #167)
`[verified]` ? so a CLI MVP has a real head start.
64     - **ffmpeg licensing (a genuine landmine):** ffmpeg is LGPL/GPL. The current stitch path
uses **libx264 (GPL)**. Redistributing that inside an installer would impose GPL on the bundle.
**Mitigation (D4):** ship an **LGPL ffmpeg build** using **openh264 (BSD)** or the OS hardware
encoder (NVENC / Apple **VideoToolbox** / Linux **VAAPI**) ? the desktop app should prefer
hardware H.264 anyway (faster, no CPU stitch tax).
65     - **Weights:** WASB-SBDT = MIT `[verified]` (`tracknet-wasb-wsl2-decision`),
redistributable.
66
67     ## 4. Design / architecture `[design]`
68
69     ### 4.1 The one-click flow (and what code each step reuses)
70     ``
71     [card inserted]
72     ? (NEW) per-OS device-insert watcher ? mount path
73     ?
74     [list videos on the card] ? glob video exts; reuse _VIDEO_EXTS from

```

```

generate_highlights.py
75      ?
76      [make on-device proxy]      ? ffmpeg downscale (hardware encoder); reuse the proxy +
--detect-from split (#205)
77      ?
78      [detect: WASB native]      ? (NEW) NativeWasbRunner (DetectorRunner subclass) ? NO WSL;
same CSV contract
79      ?
80      [windowing ? rallies]      ? REUSE trajectory_to_action_windows + improved milestone
windowing (R1: cap=6 + R4)
81      ?
82      [stitch reel from ORIGINAL] ? REUSE VideoCompiler
(backend/pipeline/stitcher/compiler.py); read original in place
83      ?
84      [export reel + tiny report to a user folder] ? reuse _atomic_publish non-destructive
write; NEVER copy originals
85      ```
86      The detector runs on a proxy; the stitch reads the original on the card (the
`--detect-from` split, already shipped [verified] in generate_highlights.py). Rally
timestamps transfer 1:1 because the proxy shares the timeline (the collector frame-identical
proxy contract, VIDEO_LOCALITY_MODEL` $1).
87
88      ### 4.2 Reuse map ? what's NEW vs REUSED
89      | Concern | Status | Code |
90      |---|---|---|
91      | Trajectory?rally windowing | REUSE [verified] |
`tracknet_runner.trajectory_to_action_windows` (model-agnostic staticmethod) |
92      | Stitch / reel compile | REUSE [verified] |
`backend/pipeline/stitcher/compiler.py::VideoCompiler` (pure ffmpeg) |
93      | Local orchestration (proxy?detect?stitch, non-destructive, resumable, `--skip-ai-
handoff`, `--detect-from`) | REUSE [verified] | tools/generate_highlights.py` |
94      | Detector runner contract | REUSE [verified] |
`backend/pipeline/detectors/base.py::DetectorRunner` ABC |
95      | WASB inference math | REUSE (lift out of WSL) [verified] |
`backend/pipeline/detectors/wasb_infer.py` |
96      | Native (de-WSL) WASB runner, per-OS | NEW [design] | NativeWasbRunner` ?
sibling of wasb_runner.py, no WslCommandMixin` |
97      | USB/SD insert detection + video listing | NEW [design] | per-OS: Win
WMI/`WM_DEVICECHANGE`, macOS DiskArbitration, Linux udev/udisks2 |
98      | App shell / one-click UI | NEW [design] | Tauri vs Electron vs CLI ($3.4; P2
decides) |
99      | Packaging + code-signing per OS | NEW [design] | PyInstaller/MSIX (Win),
.dmg+notarize (mac), AppImage/Flatpak (Linux) ? P3 |
100
101      ### 4.3 Where this sits in the platform architecture
102      This is the local` / LocalDesktop` ComputeBackend of
`PLATFORM_ARCHITECTURE.md` §3.2, realized as a shippable product instead of a server worker. The
control-plane/data-plane split degenerates cleanly to "all on one machine." Nothing here
forecloses the hosted SaaS path ? it's the privacy-max end of the same spectrum, and a genuine
differentiator (no competitor offers true on-device compute except SwingVision, which proves
it's commercially viable ? PLATFORM_ARCHITECTURE` §intro).
103
104      ### 4.4 App-shell recommendation (to confirm at P2)
105      Lean: start with a CLI + installer MVP (wraps the existing
`indexer`/`generate_highlights.py` entry point ? fastest way to validate packaging, signing,
USB-detect, and the native runner end-to-end on all three OSes), then graduate to a Tauri
GUI for the non-technical user (tiny download fits the "huge-video laptops, slim app" frame;
the Python backend runs as a sidecar the Tauri front-end calls over localhost). Electron is the
fallback if webview inconsistencies bite. This mirrors the repo's "simplest thing first,
graduate on evidence" bias.
106
107      ## 5. Risk table [design]
108      | Risk | Severity | Mitigation / where decided |
109      |---|---|---|
110      | CPU/MPS too slow on a no-GPU laptop (kills the one-click promise) | make-or-

```



```

break** | **P0 spike measures it before anything is built** ($6, $7-P0). Fallbacks: tiered
fast/slow UX, or the owned smaller model. |
111 | ffmpeg GPL contamination via libx264 in the installer | high (legal) | D4: LGPL build
+ openh264/hardware encoders only. |
112 | Code-signing friction (mac notarization mandatory; Win SmartScreen without EV cert) |
medium | P3 scopes certs/notarization per OS; budget the Apple Developer Program + a Windows
cert. |
113 | Privacy regressions (the on-device promise is the *selling point*) | medium | D1/D2:
no upload, no original-copy, no required network. Make the "nothing leaves your machine"
guarantee testable (assert no outbound calls in default mode). |
114 | Per-OS device-insert flakiness / removable-media edge cases (card pulled mid-run) |
medium | P1/P2 handle gracefully; never write to the card; fail safe. |
115 | Support burden for non-technical users (drivers, "it didn't detect my card") | medium
| favors a polished GUI + clear errors (reuse the actionable-error discipline already in the WSL
adapter). |
116
117 ## 6. Honest limits ? what this does NOT do
118 - **A DRAFT/DONE status here proves nothing about runtime feasibility.** The doc is a
scope; only the P0 numbers decide whether the product is buildable on commodity hardware.
119 - **If CPU-only is too slow on a typical laptop, the simple "wrap the pipeline" path
does not exist** ? it becomes "ship a smaller owned model" (a much larger effort, ties
`OWNED_MODEL_IMPLEMENTATION_PLAN`) or "GPU-tier only" (shrinks the addressable market). The doc
names this honestly rather than assuming the wrap is free.
120 - **No accuracy claim changes.** The desktop app inherits exactly the current rally-
detection quality (R1 milestone: precision is still the open gap per
`RALLY_DETECTION_GAP_CLOSURE.md`); packaging does not improve detection.
121 - **Gemini-quality reels need the optional online toggle.** The fully-local default uses
`--skip-ai-handoff`; users wanting the Gemini brain accept that ?1280px chunks leave the machine
(consent language, `VIDEO_LOCALITY_MODEL` OQ3).
122 - **macOS support is contingent on P0's MPS/CPU verdict** ? it is not assumed.
123
124 ## 7. Deliverables & progress tracker ? **source of truth**
125 Legend: ? Todo . ? In progress . ? Done . ? Blocked/gated. **One small PR per row**,
branched from `origin/master`, no stacking. Update the row (status + PR link) as work lands.
**Everything below is owner-gated; only this doc (the scoping PR) is in flight today.**
126
127 | ID | Deliverable | Depends on | Gated? | Status | PR |
128 |----|-----|-----|-----|-----|----|
129 | **D0** | **This scoping doc** (`DESKTOP_APP_PLAN.md` + README index) | ? | No | ? |
*(this PR)* |
130 | **P0** | **Compute-feasibility spike (make-or-break).** Measure WASB throughput for a
~10-min clip on: (a) CPU-only (torch-CPU + onnxruntime), (b) Apple MPS, (c) confirm CUDA-native
(no WSL). Output: a wall-clock table + a go/no-go verdict + "do we need a smaller model?"
recommendation. **No app code.** | D0 | **Owner greenlight** | ? | ? |
131 | **P1** | **De-WSL native detector port** (`NativeWasbRunner`, per-OS): CUDA-native
(Win/Linux), MPS/CPU fallback, identical `Frame,Visibility,X,Y` CSV; reuse windowing verbatim;
healthcheck per backend. | P0 verdict | Yes | ? | ? |
132 | **P2** | **Desktop app shell + one-click flow**: USB/SD detect ? list ? proxy ? native
detect ? stitch ? export; shell choice (CLI MVP ? Tauri) confirmed; "no upload / no original-
copy" enforced & testable. | P1 | Yes | ? | ? |
133 | **P3** | **Packaging + code-signing installers** per OS (Win MSIX/Authenticode, mac
.dmg + Developer-ID notarization, Linux AppImage/Flatpak); LGPL ffmpeg bundling; MIT/Apache/BSD
audit of the bundle. | P2 | Yes | ? | ? |
134
135 ## 8. Open questions ? owner / external
136 **Blocking gates (owner decides before the dependent phase):**
137 1. **(gates P1?P3) Greenlight P0** ? the spike is cheap and decides everything.
Recommend: yes, run P0 next.
138 2. **(gates P1) If CPU is too slow, which fork** ? tiered GPU-fast/CPU-slow UX . invest
in a smaller exported owned model (`OWNED_MODEL_IMPLEMENTATION_PLAN`) . GPU-tier-only product
(narrower market).
139 3. **(gates P2) App shell** ? confirm CLI-MVP-then-Tauri, or go straight to a GUI /
Electron.
140 4. **(gates P3) Distribution & signing budget** ? Apple Developer Program (mandatory for
notarization), a Windows code-signing cert (EV for clean SmartScreen) ? owner cost decision.

```

```

141
142     **Nice-to-knows:**
143     5. OS coverage priority for v1 (Windows-first? given the owner's box and the academy
beachhead).
144     6. Auto-update strategy (Tauri updater / Sparkle / none for the pilot).
145     7. Does the desktop app reuse the optional Gemini toggle at all, or stay strictly local
for v1?
146
147     ## 9. Definition of done
148     This subproject is DONE (? flip Status, archive) when:
149     - [ ] **P0** produced a measured throughput table (CPU / MPS / CUDA-native) and an
owner-accepted go/no-go verdict (incl. whether a smaller model is required). *(P0 alone is a
valid stopping point if the verdict is "not feasible without the owned model" ? then this doc
hands off to `OWNED_MODEL_IMPLEMENTATION_PLAN`.)
150     - [ ] **P1** the native (de-WSL) detector runs on ?1 GPU OS and ?1 CPU/MPS path,
producing byte-compatible trajectories (regression vs the WSL runner on a shared clip).
151     - [ ] **P2** a non-technical user can insert a card and get a reel with no terminal, no
upload, and no copied original ? verified end-to-end on ?1 OS.
152     - [ ] **P3** signed/notarized installers exist for the greenlit OSes; the bundle passes
an MIT/Apache/BSD (+ LGPL-ffmpeg) license audit.
153     - [ ] $7 rows all ? and the headline feasibility verdict is recorded for posterity.
154
155     ## 10. References
156     **Internal code anchors `[verified]`:**
[ `tools/generate_highlights.py` ](../tools/generate_highlights.py) (local orchestration; `--skip-
ai-handoff`, `--wasb-stream`, `--detect-from`, `_WSL_DETECTORS`, `_atomic_publish`) .
[ `backend/pipeline/detectors/base.py` ](../backend/pipeline/detectors/base.py) (`DetectorRunner`
ABC + the documented Linux-native de-risk path + `WslCommandMixin`) .
[ `backend/pipeline/detectors/wasb_runner.py` ](../backend/pipeline/detectors/wasb_runner.py) (the
WSL adapter to replace) .
[ `backend/pipeline/detectors/wasb_infer.py` ](../backend/pipeline/detectors/wasb_infer.py) (the
torch inference core to lift native) . [ `backend/pipeline/detectors/tracknet_runner.py` ](../back
end/pipeline/detectors/tracknet_runner.py) (shared `trajectory_to_action_windows`) .
[ `backend/pipeline/segmenters/wasb_hybrid.py` ](../backend/pipeline/segmenters/wasb_hybrid.py) .
`backend/pipeline/stitcher/compiler.py::VideoCompiler` . `pyproject.toml` + `indexer` console
entry (#167).
157     **Internal docs:** [ `VIDEO_LOCALITY_MODEL.md` ](VIDEO_LOCALITY_MODEL.md) (L0 fully-local
tier; OQ5 packaging) . [ `PLATFORM_ARCHITECTURE.md` ](PLATFORM_ARCHITECTURE.md) §3.2
(ComputeBackend seam) .
[ `OWNED_MODEL_IMPLEMENTATION_PLAN.md` ](OWNED_MODEL_IMPLEMENTATION_PLAN.md) (export-clean on-
device model) . [ `COMMERCIALIZATION.md` ](COMMERCIALIZATION.md) (C4 paid-LLM no-train;
MIT/Apache/BSD-only) . [ `DEFERRED_HARDENING_PLAN.md` ](DEFERRED_HARDENING_PLAN.md) (#7 packaging,
DONE) . [ `TRACKNET_WSL_SETUP.md` ](TRACKNET_WSL_SETUP.md) (the WSL setup being superseded for
shipped builds).
158     **Measured source:** `current-state.md` 2026-06-18 `COST_SCALING.md` checkpoint (WASB
~3.4× RT on RTX 2070 Super Max-Q, peak VRAM ~5.7?6.0 GB).
159     **External (to validate at P0/P2/P3):** PyTorch MPS backend . onnxruntime
(CPU/CoreML/DirectML execution providers) . Tauri vs Electron sizing . ffmpeg LGPL build /
openh264 . Apple Developer-ID notarization . Windows Authenticode/MSIX.
160
161     ---
162
163     ### Changelog
164     - `2026-06-18` ? Created. Scoping doc for the cross-platform desktop app + the de-WSL
native-compute port; all `[design]`. $7 P0 (compute-feasibility spike) named the make-or-break
gate. Owner-gated; nothing built.
165

```

11. assistant (2026-06-21T13:57:44.712Z)

DESKTOP_APP_PLAN.md confirms claim 11 (D4 at line 38, libx264 GPL, LGPL build mitigation) and claim 11's compiler.py reference. Now let me verify the khelsutra LOCAL_APPLIANCE_ARCHITECTURE.md for claim 9 (D10 supersession), check the torch version for the wasb conda env, and verify the compiler.py uses libx264.

12. user (2026-06-21T13:57:46.054Z)

```
1      # Single-box appliance ? UI-powered rally indexer + reel generator (KhelSutra Studio)
2
3      > **Status:** `DRAFT` ? owner-gated (auth choice, retention, cost ceiling, GPU host, P5
4      bridge) . **Owner:** Avi . **Created:** 2026-06-20 . **Last updated:** 2026-06-20
5      > **Lifecycle:** `DRAFT` ? IN PROGRESS ? DONE ? archived` (move to `docs/archives/` when
6      DONE)
7      > **Tracking anchors:** $9 progress tracker is the source of truth; indexed in
8      `docs/README.md`; pointer in `NEXT_STEPS.md` + memory `current-state` once work starts.
9      > **Relation:** peer-of `PER_RALLY_SELECTION.md` / `REQUEST_ACCESS_FORM.md` in this
10     repo; **consumes** the engine's decoupled `StorageBackend` + `python -m backend.worker` seams
11     and the FastAPI loop; **extends** the engine's `HOSTING_PLAN.md` gating + `UI_PROPOSAL.md` alpha
12     screens. Engine-side changes here are **proposals to coordinate** with the active
13     decoupled/worker sessions ? not written unilaterally.
14     > **Honesty note:** every engine-fact claim is tagged `[verified]` (read against engine
15     source this session), `[design]` (proposed here), or `[researched]` (needs sign-off). Not
16     legal/financial advice.
17     ---
18     ## 0. TL;DR
19     **KhelSutra Studio** turns the engine from a CLI into a **single-box, UI-powered rally
20     indexer + reel generator**: the khelsutra `web/` React SPA, served behind an **edge auth gate**,
21     drives the **existing** engine FastAPI loop (ingest ? index ? pick rallies ? reel ? download) on
22     **one machine**, with `StorageBackend=local` for the MVP. It is the **single-box deployment
23     profile of the just-shipped decoupled architecture** ? storage=local + compute=local is the
24     degenerate collapse of the same storage/worker split.
25
26     The **single most important caveat:** the engine has **zero app-layer auth and CORS
27     `***` `[verified backend/main.py:42]`, so safety is structural **only** because the engine stays
28     bound to `127.0.0.1:8000` `[verified backend/main.py:638]`, a firewall blocks inbound `8000`,
29     and a reverse proxy/tunnel is the sole authenticated ingress ? **the gate, not the app, is the
30     tenancy boundary.**
31
32     Two honest hardware truths: (1) the current droplet is **CPU-only** ? real detection
33     there is **Gemini (cloud)** or **stub (CPU mock)**; **real local-GPU detection needs a GPU
34     host** (native WASB, in-flight). (2) The owner priority is to **stand up a GPU host ASAP** ?
35     provider-agnostic (DO GPU droplet / GCE GPU VM / Cloud Run+GPU) ? but that runs **in parallel**
36     with the UI MVP, which ships on Gemini today.
37
38     ## 1. Problem & goal
39     The decoupled engine just shipped a `StorageBackend` ABC + a `python -m backend.worker
40     {infer|train}` dispatcher `[verified backend/storage/, backend/worker/__main__.py]`, and the
41     FastAPI loop already covers `process ? jobs ? query ? compile ? highlights` `[verified
42     backend/main.py]`. What's missing is a **console** that lets a non-CLI operator run the whole
43     loop on a single box, **honestly, behind a gate**.
44
45     **Headline CUJ (owner's words):** a **hosted website** where I pick an input ? **a
46     Google Drive file, a bucket file (`s3:///`/`gs:///`), or a local upload** ? it lands in the
47     **bucket (DO Spaces / GCS)** or locally ? I **generate highlights** on a **GPU-enabled host**
48     running the real e2e ? I **pick rallies, watch, and download** ? **completely from the UI.**
49
50     **Outcome ("good")** an operator signs in, picks a source, watches it index, curates
51     rallies, downloads a reel ? on the **live CPU droplet today** (Gemini/stub), and on a **GPU host
52     later** (native WASB) with **no UI fork**.
53
54     **Read to get current:** `PER_RALLY_SELECTION.md` ($3/$4/$10 contract anchors), engine
55     `HOSTING_PLAN.md` (gating), `DECOUPLED_COMPUTE/HANDOFF.md` (what's built),
56     `VIDEO_LOCALITY_MODEL.md` (privacy/retention).
57
58     ## 2. Decisions locked
59     | # | Decision | Source / date | Implication |
60     |---|---|---|---|
61     | D1 | Reverse-proxied **single origin**; engine stays `127.0.0.1:8000` | `[verified
```

```

main.py:638`; 2026-06-20 | CORS `` is moot from the browser ? the proxy is the boundary |
31 | D2 | **Edge auth**, zero engine auth code | `HOSTING_PLAN.md`; 2026-06-20 | Caddy
basicauth OR cloudflared + Cloudflare Access |
32 | D3 | MVP drives the **in-process FastAPI worker** loop, `storage=local` | `[verified
job_worker.py:37, local.py:33]`; 2026-06-20 | bucket worker = reserved scale path, out of MVP |
33 | D4 | Embed **RallyPicker** from `PER_RALLY_SELECTION.md` **verbatim** | PR [#11];
2026-06-20 | the console is a superset; do not redesign the picker |
34 | D5 | Console lives in the khelsutra **web/** React scaffold | `web/README.md`;
2026-06-20 | Cloudflare-Pages-style root, not engine static |
35 | D6 | Compute placement is a **setting**, rendered from `GET /api/capabilities` |
`[design]`; 2026-06-20 | UI offers ONLY what the box can honestly do |
36 | D7 | **Ingest is a source menu** (local upload · bucket URI · Drive) normalizing to
one video object | `[verified backend/storage/ref.py parse_uri]`; 2026-06-20 | Drive is a
**stub** today ? scoped honestly ($4.2) |
37 | D8 | GPU host is **provider-agnostic** (DO droplet / GCE VM / Cloud Run+GPU); storage
stays the bucket | Owner 2026-06-20 | compute and storage are separate planes; no provider lock-
in |
38 | D9 | **Observability is first-class** ? structured logs on every new path + a DoD item
| Owner 2026-06-20 | see $5; verified during CUJ replay |
39 | D10 | Delivery = a **self-hosted webserver + UI the user spins up** (NOT a desktop
app); the user points at **local storage + GPU/CPU OR cloud locations** and the UI drives the
tool | Owner 2026-06-20 | **supersedes `DESKTOP_APP_PLAN.md`**; one artifact (this appliance)
runs on the owner box, a droplet, or a GPU host ? same UI, compute/storage are settings |
40
41 ## 3. Foundation ? what the engine gives us today `[verified this session unless
noted]`
42 - **FastAPI loop:** `POST /api/process` (202 + `job_id`) ? poll `GET /api/jobs/{id}` ?
`POST /api/query` ? `POST /api/compile` ? `GET /api/highlights/{video_id}/{filename}`
(FileResponse) `[main.py]`. **Chunked upload exists** ? `POST /api/upload/init` · `PUT
/api/upload/{id}/part/{i}` · `POST /api/upload/{id}/complete` · `DELETE /api/upload/{id}`
`[verified uploads.py:64,89,123,147]` ? but the bundled `app.js` **never calls them** (it only
hits `/api/process` + `/api/compile` `[verified app.js:88,373]`). So browser upload is **client-
wiring only**, no engine change.
43 - **In-process worker:** one module-global `ThreadPoolExecutor(max_workers=1)`
`[verified job_worker.py:37]` ? strictly **one job at a time** ; the DB `jobs` table is the
clock; `fail_stale_jobs` sweeps queued/running on boot. Gemini chunk concurrency is a
**separate** knob `indexing.ai_max_workers` (**default 5** `[verified models.py:183]`; set to 1
to serialize) ? it is **not** already 1.
44 - **Storage:** `storage.backend` default = `local` `[verified models.py:298]`;
`LocalStorage.fetch_to_local` is **identity** `[verified local.py:33]`, `put` no-ops on
`src==dst` ? so on a single box the worker's pull?pipeline?push **collapses to read-local/write-
local**. URI scheme `parse_uri` `[verified ref.py]`: bare/local:// ? local; `s3://`
(AWS/R2/**DO Spaces**/MinIO, by endpoint) ; `gcs://` ; `gs://` ; `gdrive://` ? **follow-up stub
(emulated only)**. Credentials never in config ? boto3 default chain / `~/aws` `[verified
s3.py]`.
45 - **Compute seam:** `indexing.detector_impl` ? `{auto=WSL, native=Linux-native,
`stub`=CPU mock}; default "auto" `[verified models.py:129]`, resolved in
`trajectory_hybrid._resolve_runner` `[verified :76-85]`. **Native WASB is in-flight on branch
`feat/native-wasb-runner` : ** `native_wasb_runner.py` **exists** `[verified]`;
**`fusion_hybrid._build_native_runner` returns `NativeWasbRunner`** `[verified
fusion_hybrid.py:86]` while **`trajectory_hybrid._build_native_runner` still raises** `[verified
:57-63]`. Acceptance gate = **owner byte-parity** (WSL CSV == native CSV) on a real GPU box ?
not exercisable on the droplet or in the offline suite. Treat native as **not production-
verified**.
46 - **Worker CLI:** `python -m backend.worker {infer|train}` pulls a `--video-uri` ?
pipeline ? pushes `outputs/<video_id>/[intervals.csv,report.json]` `[verified
worker/__main__.py:88,149]`. The validated GPU-free/key-free combo: `detector_impl=stub` +
`--set indexing.skip_ai_handoff=true` + `storage.s3`.
47 - **Gating:** cloudflared Tunnel (outbound-only, no inbound ports) + Cloudflare Access
email-OTP; backend trusts `Cf-Access-Authenticated-User-Email` `[HOSTING_PLAN.md]`. `GET
/api/health` **now EXISTS** (engine M1, [#268]) ? `{status, sport, default_segmenter}`
`[verified main.py:127, corrected 2026-06-21 ? was "does not exist"]`.
48 - **Droplet reality:** `168.144.126.176`, **CPU-only, NO GPU**, region `blr1`; co-hosts
the marketing site (`/srv/khelsutra-site`, node `:8787`) **and** the engine (`~/rally`); ran the
full train+infer e2e against DO Spaces with the **GPU mocked** `[SETUP_NEW_MACHINE.md],

```

DECOUPLED_COMPUTE/HANDOFF.md]`. Shared testbed: MinIO `9000/9001` + GCS emulator `9023` also run there.

49 - **Absent** (net-new): **GET** /api/capabilities`, **GET** /api/videos` (only single-row **get_video(id)**), **GET** /api/reels/{video_id}`, any source/proxy streaming endpoint, any **Dockerfile**, any per-video **cost ledger** `[verified ? grep returns none]`.

50

51 ## 4. Design / architecture

52

53 ### 4.1 Topology & request path

54 ONE box: **edge gate** (cloudflared+Access or Caddy basicauth) ? **reverse proxy** / static host` serving the **web/`** SPA bundle and proxying **/api/`** ? engine **FastAPI** **127.0.0.1:8000`** ? **LocalStorage** ? **detector seam** (Gemini/stub today; native WASB on a GPU host). The firewall blocks inbound **8000`** + the test ports **9000/9001/9023`**. The marketing site stays a **separate** systemd unit on a **separate** subdomain.

55

56 ````

57 PUBLIC INTERNET

58 ? (1) EDGE GATE ? the ONLY ingress; auth lives HERE, not in the app

59 ? cloudflared Tunnel + Cloudflare Access (no inbound ports) OR Caddy :443 + basicauth

60 ?

13. user (2026-06-21T13:57:47.542Z)

253: "-c:v", "libx264",

14. user (2026-06-21T13:57:49.124Z)

11:| Miniconda | `~/miniconda3` |

13:| TrackNetV4 conda env | `TrackNetV4` (Python 3.9, TensorFlow 2.17 + bundled CUDA 12.3) |

15:| WASB conda env | `wasb` (Python 3.8, torch 1.11.0+cu113) ? verified

`torch.cuda.is\_available()==True` |

22:source ~/miniconda3/etc/profile.d/conda.sh

23:conda activate TrackNetV4 # or: conda activate wasb

60: "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",

61: "conda_env": "TrackNetV4",

90:source ~/miniconda3/etc/profile.d/conda.sh && conda activate TrackNetV4

15. assistant (2026-06-21T13:57:55.406Z)

Strong confirmations:

- Claim 9 (D10 supersedes DESKTOP_APP_PLAN): LOCAL_APPLIANCE_ARCHITECTURE.md:39 D10 explicitly states "supersedes `DESKTOP\_APP\_PLAN.md`" and "self-hosted webserver + UI the user spins up (NOT a desktop app)". CONFIRMED. (Cited line was :39, claim said :39 ? accurate.)

- Claim 11 (compiler uses libx264): compiler.py:253 **"-c:v", "libx264"** ? CONFIRMED libx264 (GPL) is the current encoder.

- Claim 2 (torch 1.11): TRACKNET_WSL_SETUP.md:15 ? WASB conda env = Python 3.8, torch 1.11.0+cu113. CONFIRMED exact version.

Now let me verify the external/licensing/version facts via web search. Let me batch the searches.

16. user (2026-06-21T13:58:04.918Z)

Web search results for query: "Nuitka open source license Apache-2.0 commercial license optional compiled binary distribution"

Links: [{"title": "License of Nuitka Commercial ? Nuitka the Python

Compiler", "url": "https://nuitka.net/doc/commercial-license.html"}, {"title": "FAQ using Nuitka in commercial products · Issue #245 ·

Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/245"}, {"title": "Nuitka

0.6.12", "url": "https://pypi.org/project/Nuitka/0.6.12/"}, {"title": "Nuitka/LICENSE.txt at develop

· Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/blob/develop/LICENSE.txt"}, {"title": "Nu

itka Downloads ? Nuitka the Python

```
Compiler", "url": "https://nuitka.net/doc/download.html"}, {"title": "nuitka vulnerabilities", "url": "https://security.snyk.io/package/pip/nuitka"}, {"title": "Nuitka 0.6.16.4", "url": "https://pypi.org/project/Nuitka/0.6.16.4/"}, {"title": "Nuitka 0.4.7", "url": "https://pypi.org/project/Nuitka/0.4.7/"}, {"title": "Nuitka the Python Compiler ? Nuitka the Python Compiler documentation", "url": "https://nuitka.github.io/Nuitka-website/"}]
```

Based on the search results, here's what I found about Nuitka's licensing and distribution:

Nuitka Licensing

Nuitka uses AGPL-3.0 licensing for version 4.0 and above, while Apache-2.0 was used for versions 0.4.0 through 3.9.

There is an exception to the AGPLv3 for the created binaries, which are not bound by the AGPLv3 requirements for the user's code, similar to how GCC operates.

Commercial License Option

For commercial support and advanced features, Nuitka Commercial is available. The commercial license forbids the distribution of its source code or binaries. However, this does not impact your software at all, nor how you use compiled binaries of your software?you are free to distribute those without any restrictions.

The commercial edition additionally protects your code, data and outputs, so that users of the executable cannot access these.

Compiled Binary Distribution

The key point regarding compiled binary distribution is that the created binaries are not bound by the AGPLv3 requirements for the user's code, meaning you can freely distribute compiled executables regardless of which license version of Nuitka you use.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

17. user (2026-06-21T13:58:05.831Z)

Web search results for query: "PyInstaller license GPL bootloader exception closed source application distribute"

```
Links: [{"title": "License ? PyInstaller 6.20.0 documentation", "url": "https://pyinstaller.org/en/stable/license.html"}, {"title": "Pyinstaller license issue", "url": "https://groups.google.com/g/pyinstaller/c/dDXwMMnAtQ8/m/-DdGuiNoBAAJ"}, {"title": "License ? PyInstaller 4.2 documentation", "url": "https://pyinstaller.org/en/v4.2/license.html"}, {"title": "COPYING.txt - Git Hub", "url": "https://raw.githubusercontent.com/pyinstaller/pyinstaller/develop/COPYING.txt"}, {"title": "LGPL and GPL License Compliance with PyInstaller - velovix", "url": "https://velovix.github.io/post/lgpl-gpl-license-compliance-with-pyinstaller/"}, {"title": "PyInstaller Documentation Release 6.20.0 David Cortesi 2026-06-03", "url": "https://pyinstaller.org/_/downloads/en/latest/pdf/"}, {"title": "pyinstaller · PyPI", "url": "https://pypi.org/project/pyinstaller/3.4/"}, {"title": "Pyinstaller prunes license text of included libraries and packages · Issue #5666 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/5666"}, {"title": "Licenses, esp GPL readline with onefile · Issue #4471 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/4471"}]
```

Based on the web search results, here's what I found regarding PyInstaller's licensing:

PyInstaller License Overview

PyInstaller is distributed under a dual-licensing scheme using both the GPL 2.0 License, with an exception that allows you to use it to build commercial products, and the Apache License,

version 2.0, which only applies to a certain few files.

Bootloader Exception

PyInstaller is distributed under the terms of the GNU General Public License (version 2 or later) with exception for distributing the bootloader. The SPDX identifier for this is "GPL-2.0-or-later WITH Bootloader-exception".

Commercial and Closed-Source Applications

You may use PyInstaller to bundle commercial applications out of your source code. More specifically, the executable bundles generated by PyInstaller from your source code can be shipped with whatever license you want, as long as it complies with the licenses of your dependencies.

Important Caveats

It's important to note that while PyInstaller itself has this exception for building commercial and proprietary applications, there are compliance concerns: PyInstaller is a very aggressive packaging tool and pays no mind to the licenses of the libraries it packages up, which poses a significant challenge if your project does not use a GPL-compatible license, because you may be violating the licenses of your LGPL or GPL dependencies.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

18. assistant (2026-06-21T13:58:09.957Z)

Critical finding for Claim 4 (Nuitka license). The search reveals Nuitka is NOT simply Apache-2.0 as the claim states. Nuitka switched to **AGPL-3.0** for version 4.0+ (Apache-2.0 was only versions 0.4.0-3.9). The claim says "Open-source Nuitka core is Apache-2.0 (PyPI)" with high-confidence framing ? this is now likely OUTDATED/REFUTED for current versions. Let me verify the current Nuitka version on PyPI and its license precisely.

19. user (2026-06-21T13:58:13.765Z)

Nuitka Version and License

Current Version: 4.1.3 (released June 21, 2026)

License: The documentation states the license field as:

"GNU Affero General Public License v3 or later (AGPLv3+)"

This is **AGPL**, not Apache-2.0. The full license text is available at <https://www.gnu.org/licenses/agpl-3.0.html>, with a runtime exception provided in the LICENSE-RUNTIME.txt file.

20. user (2026-06-21T13:58:16.879Z)

Nuitka License Analysis

Based on the provided content, **Nuitka uses the GNU Affero General Public License v3 (AGPL-3.0)**, not Apache-2.0.

Regarding compiled binaries, the license does not provide a blanket exception allowing free distribution of compiled output. The AGPL-3.0 requires that when you convey a covered work in object code form, you must also provide "the machine-readable Corresponding Source under the terms of this License."

However, there is one potential avenue: if someone distributes a compiled binary as part of a

"User Product" (defined as consumer products or items designed for incorporation into dwellings), specific installation requirements apply. But this is a narrow exception and doesn't constitute a general carve-out for freely distributing compiled binaries without source code obligations.

The key distinction of AGPL-3.0 over the standard GPL is its network server provision: modified versions deployed on public servers must make source code available to users, which is stricter than typical open-source licenses.

21. assistant (2026-06-21T13:58:22.688Z)

Confirmed: Nuitka 4.1.3 (current, June 2026) is **AGPL-3.0**, NOT Apache-2.0. Apache-2.0 only applied to old versions 0.4.0-3.9. Claim 4 is REFUTED as stated (it claims "Open-source Nuitka core is Apache-2.0 (PyPI)" as a present-tense fact).

However, the second part of claim 4 ? that compiled output doesn't inherit Nuitka's license and Commercial is optional ? needs nuance. There IS a runtime exception (LICENSE-RUNTIME.txt) so your compiled program's own license isn't forced to AGPL. Let me check the Nuitka commercial-license page and the runtime exception to assess this precisely, and also verify the PyInstaller PyTorch hidden-imports and AV claims.

22. user (2026-06-21T13:58:27.244Z)

Based on the provided content, I can only address your first question:

Nuitka Commercial is optional. The page states that "you are free to distribute [compiled binaries] without any restrictions" regardless of which version you use. This indicates that purchasing the commercial license is not required to distribute compiled binaries.

However, the document doesn't contain information about:

- Open-source Nuitka's specific licensing terms regarding source code distribution
- Any runtime exceptions
- PyTorch or onnxruntime support

To answer these remaining questions accurately, I would need additional documentation beyond what was provided.

23. user (2026-06-21T13:58:31.392Z)

Web search results for query: "PyInstaller torch hidden imports copy-metadata torch._C._jit nvrnc onefile file already exists pyd"

Links: [{"title": "Simplifying Deployment: Using PyInstaller with PyTorch ? codegenes.net", "url": "https://www.codegenes.net/blog/pyinstaller-pytorch/"}, {"title": "Detect usages of torch.jit and collect its required source files automatically · Issue #6290 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/6290"}, {"title": "Pyinstaller problem with a module that uses torch and pytorch_lightning. · pyinstaller · Discussion #7944", "url": "https://github.com/pyinstaller/pyinstaller/discussions/7944"}, {"title": "Pyinstaller and Pytorch · pyinstaller · Discussion #7621", "url": "https://github.com/pyinstaller/pyinstaller/discussions/7621"}, {"title": "Unable to import torch in an exe · Issue #8348 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/8348"}, {"title": "Problem exporting gpytorch (jit) · Issue #7647 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7647"}, {"title": "Changelog for PyInstaller ? PyInstaller 6.14.2 documentation", "url": "https://pyinstaller.org/en/v6.14.2/CHANGES.html"}, {"title": "Understanding PyInstaller Hooks ? PyInstaller 5.13.0 documentation", "url": "https://pyinstaller.org/en/v5.13.0/hooks.html"}, {"title": "Bug pyinstaller+HTTPServer/flask+torch model prediction: not executing · Issue #8210 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/8210"}]

Based on the search results, here's what I found about using PyInstaller with PyTorch and the specific configuration options you mentioned:

PyInstaller with PyTorch Hidden Imports

PyInstaller can be configured with hidden imports for PyTorch using `--hidden-import=torch._C._jit --hidden-import=torch._C._nvrtc` flags along with the `--onefile` option.

Collecting Metadata

To handle packages like transformers that require metadata from the torch package, you need to instruct PyInstaller to collect the metadata by adding `--copy-metadata torch` to your PyInstaller command.

JIT and Source File Collection

Warnings about JIT are caused by lack of source .py files for JIT, as PyInstaller does not collect source .py files by default, and this can be addressed by modifying your spec file to use `module_collection_mode = { 'torchvision': 'pyz+py' }` to ensure that source .py files are collected.

Recent versions of PyInstaller implement discovery of source .py files for modules, which allows collection and discovery of source .py files that might be required by frameworks that perform aggressive recursive introspection, such as torch JIT.

These configurations help resolve common issues when packaging PyTorch applications with PyInstaller, particularly around the ".pyd file already exists" errors you may encounter when using the `--onefile` mode.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

24. assistant (2026-06-21T13:58:34.760Z)

Claim 8 (torch hidden imports / copy-metadata torch / file already exists) is CONFIRMED by the search. Now let me verify the AV/false-positive claim (5), the macOS notarization claim (6), the uvicorn PyInstaller pattern (1), and the Nuitka build-time / torch pruning claim (7).

25. user (2026-06-21T13:58:42.223Z)

Web search results for query: "PyInstaller onefile antivirus Windows Defender false positive onedir self-extract temp code signing"

Links: [{"title": "My --onefile exe is getting anti-Virus False positive flags and is disabled by windows. · Issue #6754 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/6754"}, {"title": "How to Fix Antivirus False Positives with PyInstaller Executables", "url": "https://www.pythonguis.com/faq/problems-with-antivirus-software-and-pyinstaller/"}, {"title": "Pyinstaller - false positive? - Python Help - Discussions on Python.org", "url": "https://discuss.python.org/t/pyinstaller-false-positive/43171"}, {"title": "anti-virus says exe is a threat · pyinstaller · Discussion #5877", "url": "https://github.com/orgs/pyinstaller/discussions/5877"}, {"title": "Compacting using pyinstaller causing antivirus false positives · Issue #711 · upx/upx", "url": "https://github.com/upx/upx/issues/711"}, {"title": "How to stop your Python programs being seen as malware | by Mark Hank | Medium", "url": "https://medium.com/@markhank/how-to-stop-your-python-programs-being-seen-as-malware-bfd7eb407a7"}]

Based on the search results, here's what I found regarding PyInstaller, antivirus false positives, and Windows Defender:

Why PyInstaller Causes False Positives

PyInstaller's `--onefile` mode creates a single executable that extracts everything to a temporary folder, and this extraction-and-execution pattern is exactly what a lot of actual malware does, as malicious software often uses "packers" to compress and hide their payloads, extracting them at runtime to avoid detection. Because PyInstaller's behavior looks similar from the outside, antivirus heuristics can flag PyInstaller-built executables as suspicious.

Additionally, PyInstaller is genuinely used by some malware authors, and the PyInstaller bootloader has ended up on some AV threat lists.

Recommended Solutions

Solutions include updating PyInstaller to the latest version (or compiling the bootloader yourself), using one-folder mode and distributing with a proper installer, code-signing your executable with a valid certificate, and submitting false positive reports to the AV vendors.

Code signing doesn't make false positives impossible, but it dramatically reduces them, especially with Windows Defender and SmartScreen.

You can submit your file with a short note explaining why you think it's a false positive to Microsoft.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

26. user (2026-06-21T13:58:43.932Z)

Web search results for query: "macOS notarization PyInstaller hardened runtime entitlements allow-jit unsigned-executable-memory disable-library-validation notarytool stapler deep discouraged"

Links: [{"title": "Hardened Runtime Code Signing for the macOS .app bundle", "url": "https://groups.google.com/g/pyinstaller/c/ck5zLIwbmw"}, {"title": "Signing and notarizing a Python MacOS UI application", "url": "https://haim.dev/posts/2020-08-08-python-macos-app"}, {"title": "macOS binaries crash if codesigned with hardened runtime enabled · Issue #4629 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/4629"}, {"title": "Notarized software still gets labe? | Apple Developer Forums", "url": "https://developer.apple.com/forums/thread/132908"}, {"title": "Notarization, hardened runtimes, LCB, and executables - LiveCode Forums.", "url": "https://forums.livecode.com/viewtopic.php?t=33633"}, {"title": "Gatekeeper rejects notarized app (? | Apple Developer Forums", "url": "https://developer.apple.com/forums/thread/794080"}, {"title": "How to notarize an MacOS command line tool created outside of XCode", "url": "https://www.codeprintr.com/thread/2423384.html"}, {"title": "Notarization: the hardened runtime ? The Eclectic Light Company", "url": "https://eclecticlight.co/2021/01/07/notarization-the-hardened-runtime/"}]

Based on the web search results, here's what I found regarding macOS notarization with PyInstaller and the related entitlements and tools you mentioned:

Key Findings

****Hardened Runtime & Entitlements:****

To notarize a PyInstaller app, you must use the hardened runtime, and for Python applications specifically, you need to allow unsigned executable memory. The main entitlements required include ``com.apple.security.cs.allow-unsigned-executable-memory`` set to true in a separate entitlements.plist file.

****Additional Entitlements for Complex Apps:****

For more complex applications with embedded components, additional entitlements may be needed:

- ``com.apple.security.cs.allow-jit`` and ``com.apple.security.cs.allow-unsigned-executable-memory`` are required for binaries built by PyInstaller.
- ``com.apple.security.cs.disable-library-validation`` can be used as an entitlement when signing executables built with PyInstaller.

****Code Signing:****

Apple recommends not using "deep-sign", but in the case of PyInstaller apps it's actually

required as all the bundled Python libraries need to be signed. Adding the `--timestamp` parameter includes a secure timestamp with the code-signing signature, which is a requirement for notarization.

****Important Cautions:****

You should only add entitlements to executables; adding them to non-executables can be actively harmful. Recent versions of macOS and the notary service have become stricter about not including comments in entitlements files.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

27. assistant (2026-06-21T13:58:48.980Z)

Claim 5 (AV/onefile/onedir/code-signing) is CONFIRMED. Claim 6 is mostly confirmed (hardened runtime + the three entitlements + notarytool + stapler), but there's a nuance: one source says `--deep` is "actually required" for PyInstaller apps because all bundled libs need signing, which partially conflicts with the claim's `--deep` discouraged, sign inside-out." Apple's general guidance does discourage `--deep`, but for PyInstaller specifically it's contested. Let me verify the uvicorn/PyInstaller pattern (claim 1) and the Nuitka build-time/torch pruning (claim 7), plus a quick check on the bundle-size claim (4/size).

28. user (2026-06-21T13:58:55.744Z)

Uvicorn with PyInstaller Recommendations

Based on the discussion content provided, there is ****no official recommended guidance**** explicitly stated about running uvicorn under PyInstaller when frozen.

However, the discussion illustrates a real problem: when using PyInstaller with uvicorn's multi-worker mode (`num_workers > 1`), some worker processes fail to start with `OSError: [WinError 10022] An invalid argument was supplied`.

The user's code shows they are:

- Using `freeze_support()` from multiprocessing
- Passing an import string ("rest:app") rather than an app object
- Setting `reload=False`
- Attempting to use `workers=4`

The issue occurs intermittently (~50% failure rate) with only 2-4 workers failing to spawn, suggesting a race condition or resource contention specific to PyInstaller's frozen executable environment.

****The discussion remains unanswered****, so no definitive best practices are provided in this thread. To find official guidance, you would need to consult uvicorn's documentation or filed issues/PRs directly addressing PyInstaller compatibility.

29. user (2026-06-21T13:58:58.146Z)

Web search results for query: "Nuitka build time slow torch standalone compile minutes onnxruntime plugin operator kernel pruning reduce size"

Links: [{"title": "Very long compile times for pytorch standalone build · Issue #754 · Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/754"}, {"title": "Compile time, created files size, and exec time all seem large (Win10/mingw64 in conda) · Issue #599 · Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/599"}, {"title": "Nuitka | pydevtools", "url": "https://pydevtools.com/handbook/reference/nuitka/"}, {"title": "Speed up Nuitka Compilation time - CodersLegacy", "url": "https://coderslegacy.com/python/speed-up-nuitka-compilation-time/"}, {"title": "Troubleshooting performance issues - onnxruntime", "url": "https://onnxruntime.ai/docs/performance/tune-performance/troubleshooting.html"}, {"title": "ONNX Runtime | onnxruntime", "url": "https://onnxruntime.ai/docs/"}, {"title": "Nuitka takes extremely long time to"}]

compile · Issue #1397 ·

```
Nuitka/Nuitka", "url": "https://github.com/Nuitka/Nuitka/issues/1397"}, {"title": "Nuitka takes extremely long time to compile", "url": "https://lightrun.com/answers/nuitka-nuitka-nuitka-takes-extremely-long-time-to-compile"}]
```

The search results provide relevant information about Nuitka compilation times and optimization strategies:

Key Findings:

****Build Time Issues:****

Compilation is slower than bundling, with medium-size applications taking 5 to 15 minutes in Nuitka because every module passes through a C compiler. Users have reported compile times of tens of minutes, with build/dist folders reaching 926 megabytes.

****Kernel Pruning and Optimization:****

Recent releases added built-in package-specific rules for PyTorch, TensorFlow, scikit-learn, and SciPy, with rules that prune unused operator kernels and skip compiling TensorFlow's generated code, which was previously a build-time blocker.

****Compilation Speed Optimization:****

Several strategies can help reduce build times:

- The cache speeds up compilation by re-using previously compiled build data; when recompiling after changes, about 80-90% of code remains the same, so only 10-20% needs recomputation, drastically speeding up recompilation times.
- Anti-bloat is a smarter way of checking for imports and removing the ones which are not needed by your program.

****Plugin Considerations:****

ML projects still need the right plugin enabled for each framework, and not every model-serving stack compiles cleanly on the first try.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. assistant (2026-06-21T13:59:05.516Z)

Important findings:

****Claim 1 (uvicorn discussion 1820):**** The cited source (discussion #1820) does NOT actually contain "PyInstaller-recommended pattern" guidance ? it's an UNANSWERED problem thread about workers>1 failing. The user in that thread was using an import string AND freeze_support AND workers=4. So the claim mischaracterizes the source as recommending a pattern. The underlying engineering facts (app object, reload=False, workers=1 avoid the multiprocessing pitfalls) are reasonable and the PyInstaller common-pitfalls page does discuss multiprocessing/freeze_support ? but the specific source citation as a "recommended pattern" is not supported, AND the engine does NOT call freeze_support() (verified absent). So claim 1 is partially refuted.

****Claim 7 (Nuitka build time + torch pruning):**** CONFIRMED ? 5?15 min builds, and built-in PyTorch rules that prune unused operator kernels are real (recent releases). Good.

Let me verify the PyInstaller common-pitfalls page for the multiprocessing/freeze_support recommendation and confirm the bundle-size claim, to finalize claims 1 and 4.

31. user (2026-06-21T13:59:12.470Z)

PyInstaller's Guidance on multiprocessing.freeze_support()

Yes, PyInstaller strongly recommends calling `multiprocessing.freeze\_support()` in frozen applications. The documentation states that it is "required to be called on all platforms."

Key Recommendations

****When to call it:****

The guidance indicates that the function "should be called before trying to use any of multiprocessing functionality (such as spawning a process or opening process pool, or allocating a shared resource, for example a semaphore)."

****The entry point pattern:****

PyInstaller's recommended approach uses the standard pattern:

```
> "a typical call looks like this: from multiprocessing import Process, freeze_support if
__name__ == '__main__': freeze_support() Process(target=f).start()"
```

****Early calling in specific scenarios:****

The documentation notes that in some cases, you may need to call it even before top-level imports, particularly if imported modules use multiprocessing, parse command-line arguments, or initialize GUI frameworks during import.

Critical Issue Without It

Without this call, the documentation warns of "an endless spawn loop of your application process" because spawned worker processes will re-execute your main program code rather than their intended multiprocessing functionality, creating recursive spawning.

32. user (2026-06-21T13:59:14.885Z)

Web search results for query: "PyInstaller pytorch opencv bundle size hundreds of MB 1GB CUDA wheels exclude torchvision matplotlib scipy onnxruntime smaller"

Links: [{"title": "PyInstaller, issue to build a package with CUDA, CuDNN and Tensorflow on Windows 10 · Issue #7175 · pyinstaller/pyinstaller", "url": "https://github.com/pyinstaller/pyinstaller/issues/7175"}, {"title": "PyTorch 2.0 distribution that uses cuda only if available? - PyTorch Forums", "url": "https://discuss.pytorch.org/t/pytorch-2-0-distribution-that-uses-cuda-only-if-available/177230"}, {"title": "Reducing the Size of PyTorch-Based Executable · pyinstaller · Discussion #8552", "url": "https://github.com/orgs/pyinstaller/discussions/8552"}, {"title": "Missing Linux x86-64 Large wheels (cuda bundled) -- can not install alongside TensorFlow - PyTorch Forums", "url": "https://discuss.pytorch.org/t/missing-linux-x86-64-large-wheels-cuda-bundled-can-not-install-alongside-tensorflow/197651"}, {"title": "Changelog for PyInstaller ? PyInstaller 6.13.0 documentation", "url": "https://pyinstaller.org/en/v6.13.0/CHANGES.html"}]

Based on the search results, here's what I found regarding PyInstaller, PyTorch, and bundle size optimization:

Key Findings

****PyTorch Bundle Sizes with CUDA:****

PyTorch 2.0 without CUDA dependencies has a wheel size of ~620MB, while PyTorch 2.0 with bundled CUDA 11.7 has a wheel size of 1.8GB.

****Challenges with PyInstaller:****

The original torch2.3+cu118 in conda is around 5GB, and the torch library in the app is about 4GB. Most of that size are binary extensions and CUDA/cuDNN shared libs, which cannot be split.

****Size Reduction Strategy:****

To achieve smaller executables, the approaches are either using CPU-only torch builds (no CUDA), or some older builds with older (and smaller) CUDA libs.

****PyTorch Wheel Changes:****

Large wheels for PyTorch > 2.2 have been replaced by small wheels from PyPI, where versions of CUDA libraries are hardcoded.

The search results indicate that achieving significantly smaller bundles with CUDA-enabled PyTorch is challenging due to the inherent size of CUDA and cuDNN libraries that are bundled with PyTorch wheels.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

33. assistant (2026-06-21T13:59:21.781Z)

This confirms the key nuance on claim 1: PyInstaller DOES recommend `freeze\_support()`, and the engine code does NOT call it. So the claim's parenthetical "already matching the PyInstaller-recommended pattern (reload=False, workers=1, freeze_support())" is inaccurate on the freeze_support() component ? the code does not have it, and that's a real gap (the executor uses ThreadPoolExecutor, not multiprocessing, so the endless-spawn loop risk is lower, but the claim's "already matching" is overstated).

Claim 4 (size): The torch+CUDA wheel figures (620MB CPU-only, 1.8GB with CUDA, even 4-5GB observed) actually exceed the claim's "250MB-1GB+" range on the high end, but the claim does say "CUDA wheels push higher," so it's directionally correct. The mitigations (headless opencv, exclude torchvision/matplotlib/scipy, ONNX/onnxruntime) are all valid and corroborated.

Let me confirm the PyInstaller license precisely (claim 3) and quickly check the executor type in main.py to be precise about the multiprocessing risk for claim 1.

34. user (2026-06-21T13:59:25.179Z)

```
backend\main.py:75:# `backend.main` (`JobWaiting`, `_drain_due_jobs`, `_arm_wake_timer`,
`_get_executor`,
backend\main.py:84:     _get_executor,
backend\main.py:304:     _get_executor().submit(_drain_due_jobs)
backend\main.py:734:         _get_executor().submit(_drain_due_jobs)
backend\api\job_worker.py:10:`_arm_wake_timer`, `_get_executor`, `_MAX_DRAIN_WAKE_SEC` ? is
resolved THROUGH the
backend\api\job_worker.py:21:from concurrent.futures import ThreadPoolExecutor
backend\api\job_worker.py:30:_job_executor: Optional[ThreadPoolExecutor] = None
backend\api\job_worker.py:33:def _get_executor() -> ThreadPoolExecutor:
backend\api\job_worker.py:35:     global _job_executor
backend\api\job_worker.py:36:     if _job_executor is None:
backend\api\job_worker.py:37:         _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
backend\api\job_worker.py:38:     return _job_executor
backend\api\job_worker.py:177:     timer = threading.Timer(delay_sec, lambda:
_main._get_executor().submit(_main._drain_due_jobs))
backend\eval\gemini_refine.py:121:     with
concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
backend\eval\gemini_refine.py:186:     with
concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as ex:
```

[Showing results with pagination = limit: 15]

35. user (2026-06-21T13:59:26.267Z)

PyInstaller License Summary

PyInstaller uses a **dual-licensing scheme**: GPL 2.0 (with commercial exceptions) and Apache License 2.0 (for specific files).

Commercial Applications

Yes, you can ship closed-source/proprietary applications built with PyInstaller without distributing its license or source code.

The relevant exception states:

> "You may use PyInstaller to bundle commercial applications out of your source code."

Additionally, the documentation confirms that "The executable bundles generated by PyInstaller

from your source code can be shipped with whatever license you want, as long as it complies with the licenses of your dependencies."

****Key point**:** You don't need to include PyInstaller's license or provide acknowledgments with your application?only ensure your dependencies' licenses are respected.

36. assistant (2026-06-21T13:59:33.446Z)

This refines claim 1's freeze_support nuance: the engine uses `ThreadPoolExecutor` (not `ProcessPoolExecutor`/`multiprocessing`), so the endless-spawn-loop hazard that `freeze\_support()` guards against does not apply to the engine's own job executor. The claim asserting the code "already matches" a pattern that includes freeze_support() is doubly off: (a) freeze_support() is not present, and (b) it isn't even needed given threads. But the load-bearing engineering point (single-process uvicorn, app object, 127.0.0.1, no workers/reload ? which is what actually makes it freeze-friendly) IS verified at main.py:735.

I have enough to render verdicts on all 10 claims. Let me confirm claim 3 is fully verified (it is) and finalize. One last note on claim 6's `--deep` nuance is already captured. Let me submit the structured output.

37. user (2026-06-21T14:00:57.574Z)

Structured output provided successfully

38. assistant (2026-06-21T14:01:04.206Z)

I've completed the adversarial verification of all 10 claims. Summary of verdicts:

- ****Confirmed (7):**** Claims 2, 3, 5, 6, 7, 8, 9 ? all verified against code and/or current web sources.
- ****Refuted (1):**** Claim 4 ? Nuitka is ****AGPL-3.0**** (v4.1.3, current), NOT Apache-2.0. Apache-2.0 only covered old versions 0.4.0?3.9. The compiled-output-doesn't-inherit and Commercial-is-optional sub-parts are correct (GCC-style runtime exception), but the headline license label is wrong.
- ****Uncertain (2):**** Claim 1 ? the freeze-friendly code facts (app object, 127.0.0.1, no workers/reload at main.py:735) are real, but the engine does NOT call `freeze\_support()` (it's absent), and the cited uvicorn discussion #1820 is an unanswered bug thread, not a "recommended pattern." Mitigated by the runner being a ThreadPoolExecutor, not multiprocessing.

The recommendation's overall shape (CLI/server appliance, tiered, PyInstaller onedir not onefile, defer WASB to sidecar-or-ONNX) is sound and well-supported, but the two factual defects above should be corrected before it's relied on for a licensing audit.